Real-Time Fleet Monitoring Platform

Big Data Frameworks I (ST2DEI-DE) · Capstone project · Instructor: Ralph Bou Nader
Nathan CHEVAUCHE · Benjamin TONNEAU · EFREI Paris, promo 2028 · November 2026

1. What we built

A platform that ingests the telemetry of 40 delivery vehicles with Kafka, validates, deduplicates and enriches it with Spark Structured Streaming, raises alerts and regional KPIs in real time, and produces a daily report with Spark batch jobs orchestrated by Airflow, only when the day's data passes a quality gate. Everything starts with `docker compose build` and `docker compose up -d` (one Kafka broker in KRaft mode, an init container that creates the six topics and publishes the catalog, the provided simulator, two notifier instances, the streaming job and Airflow standalone). Stack: Python, PySpark 3.5.1, confluent-kafka 2.5, Airflow 2.9.3, as required.

Changes to the provided material, all minimal: (1) an internal listener `kafka:29092` in the Compose file, because the broker only advertised `localhost:9092`, which other containers cannot reach; (2) the Kafka address of `create_topics.py` and of the simulator read from an environment variable (default unchanged); (3) a bug fix in `create_topics.py`, which crashed when the topics already existed, so the second `docker compose up` failed. The fault injection of the simulator is untouched.

2. Architecture

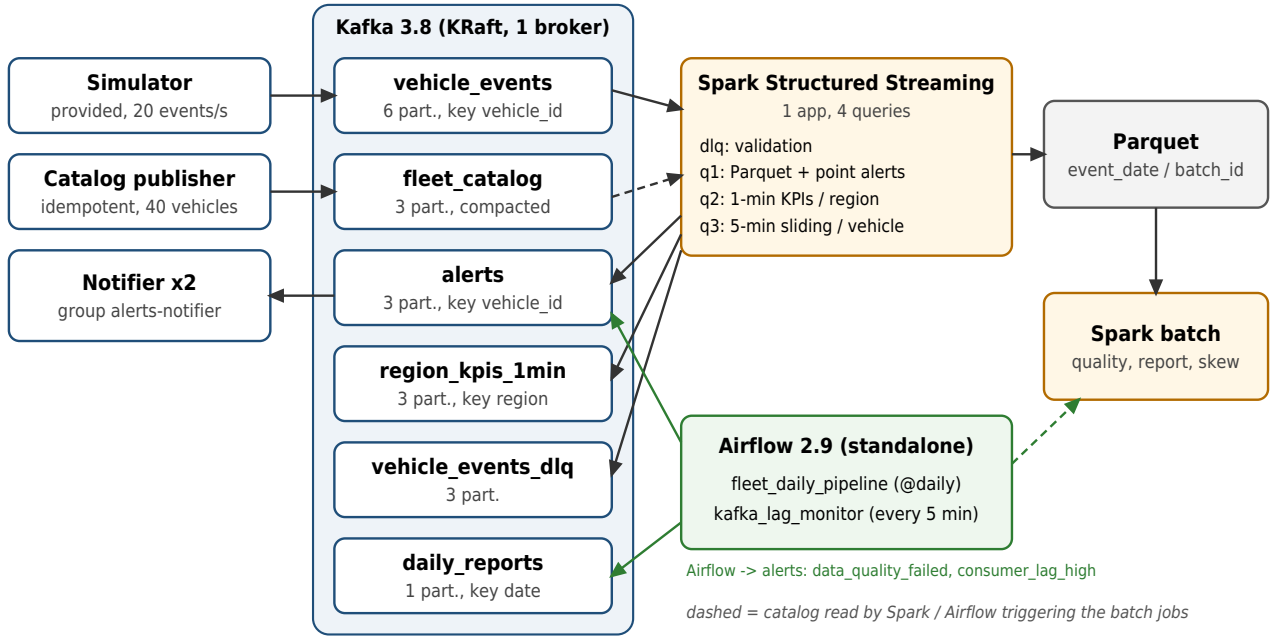


Figure 1. Data flow. Plain arrows: data; dashed: catalog read by Spark, Airflow triggering the Spark batch jobs, Airflow reading the committed offsets of the notifier group.

One Spark application runs four streaming queries, each with its own checkpoint. The Parquet files are only the hand-off between the streaming side and the batch side. All our code shares one image (Python 3.11 + Java 17); Airflow has its own image with Java and PySpark so that its tasks can call `spark-submit`.

3. Kafka layer (Part A)

Catalog and compaction

The 40 catalog records are published with an idempotent producer (`enable.idempotence=True`: a producer id and a sequence number per partition let the broker drop a retried duplicate). The random fields use a fixed seed so that republishing gives the same catalog. Compaction guarantees that at least the latest value of every key is kept, so a consumer reading from the beginning rebuilds the current catalog. It does not guarantee when old values disappear: the active segment is never compacted, a segment only closes

at 1 GB or after `segment.ms` (7 days by default), and the cleaner waits for `min.cleanable.dirty.ratio` (0.5). With a catalog of a few KB, older versions can stay for days, so our Spark job keeps the record with the highest offset per key and treats a null value (tombstone) as a removal.

Notifier

Group `alerts-notifier`, `enable.auto.commit=False`, synchronous commit after the alert is printed, `cooperative-sticky` assignment. Measured on the laptop: after a `docker kill` of one of the two instances, the survivor received its partitions 43 s later (session timeout 45 s, a SIGKILL sends no LeaveGroup) without giving up its own; on restart, in our first run only the moved partition went back. In a second run a commit fell inside the rebalance and failed with `ILLEGAL_GENERATION`: the first version of the notifier crashed on it, we now catch the exception and continue (the new owner of the partition reads the alert again: at-least-once).

Keys, partitions and delivery semantics

Topic	Part.	Key	Why
vehicle_events	6	vehicle_id	order per vehicle; 6 = ceiling of read parallelism (one Spark task and at most one consumer per partition)
fleet_catalog	3	vehicle_id	compaction works per key: without a key there is nothing to compact
alerts	3	vehicle_id	at least 2 so both notifiers work, alerts of a vehicle stay in order
region_kpis_1min	3	region	updates of a (region, window) stay in order, the last one wins
vehicle_events_dlq	3	original key	low volume
daily_reports	1	date	one message per day, total order

A random key on `vehicle_events` would spread the events of a vehicle over 6 partitions: no order per vehicle any more, and the two copies of a duplicate could land on different partitions (Spark's dedup still works because it shuffles by `event_id`). One partition would give total order but a single leader, one Spark read task and one active consumer per group: no scalability.

Hop	Semantics	Reason
producer -> broker	exactly-once per partition for retries	idempotent producer; the simulator's deliberate duplicates are real distinct messages, removed by Spark
broker -> Spark	exactly-once for the state	Spark stores the offsets in its checkpoint (not in a consumer group); replayable source
Spark -> Parquet	exactly-once	a replayed batch overwrites its own `batch_id` folder (Section 4)
Spark -> alerts / KPIs / DLQ	at-least-once	the Kafka sink appends; a replayed batch publishes again
alerts -> notifier	at-least-once	commit after handling: a crash between print and commit re-prints the alert

4. Spark Structured Streaming (Part B)

- **Validation and DLQ.** Explicit schema; `event_time` read as a string then converted, so a bad timestamp becomes NULL and is detectable. `from_json` does not return NULL on broken JSON but a struct of NULL fields, so we use `columnNameOfCorruptRecord`. The reason is a chain of `when` where NULL tests come first (a comparison with NULL is neither true nor false, so `speed > 250` alone would let a NULL speed pass as valid), and the split is done on `reason IS NULL` / `IS NOT NULL`, which are never NULL: every message goes to exactly one branch. Over the whole run: 10240 invalid inputs, 10240 DLQ messages with their reason.
- **Deduplication** with `withWatermark('event_time', '2 minutes').dropDuplicatesWithinWatermark(['event_id'])`. An id stays in the state only while its event time is above the watermark (max event time seen minus 2 min), so the state holds about 2 minutes of events. A duplicate arriving after the watermark is dropped as late data. Consequence: the simulator's late events (3 to 6 min) are dropped too, about 2 % of the traffic (19995 events over our run).
- **Enrichment.** Batch read of the compacted topic, last record per key, tombstones removed, `broadcast` and a left join, then `coalesce(home_region, 'UNKNOWN')`. The static side is not cached, so Spark re-reads the catalog at every micro-batch: after a tombstone on V0040 its next events were UNKNOWN without restart. Limitations: the change only applies from the next micro-batch, events already written keep the old region, the whole topic is re-read at every batch, and each query reads it at its own moment.

- **Query 1** (`foreachBatch`): Parquet partitioned by `event_date` and `batch_id` with `mode('overwrite')` and `partitionOverwriteMode=dynamic`: a replayed batch has the same offsets, so the same rows and the same `batch_id`, and rewrites only its own folder (tested: same 234 rows, total unchanged). Without `dynamic`, overwrite would erase the whole folder. Point alerts (speed > 120 km/h or harsh braking) go to `alerts` with a Kafka write, which only appends: replaying a batch republishes its alerts.
- **Query 2**: 1-min tumbling window per region (count, mean and max speed, speeding events), update mode, key = region. **Query 3**: 5-min windows sliding every minute per vehicle, `sustained_speeding` when the mean is above 100 km/h over at least 5 events. The simulator alone almost never triggers it (random speed per event), so a separate demo script sends one vehicle at 108-118 km/h. Update mode gives an immediate alert but repeats it at every batch where the window changes; append mode would give one alert per window but up to 7 min later. We kept immediacy.
- **Checkpoints.** One per query: `offsets/` (planned offsets of each batch), `commits/` (finished batches), `state/` (dedup and aggregation state), `metadata`. On restart Spark resumes after the last commit and replays the batch that was planned but not committed. Deleting one checkpoint restarts only that query from the earliest offset: tested on q1, the whole history was reprocessed into a new `batch_id=0` while the old folders stayed (44403 rows for 22758 events) and every old alert was published again.
- `spark.sql.shuffle.partitions=6` instead of 200: with 200, every stateful operator runs 200 tasks and 200 state stores for a few hundred rows per batch (Section 8).

5. Spark batch (Part C)

Quality job. Rows, duplicated event_ids (rows minus distinct ids) and share of UNKNOWN for one date, written to `data/quality/<date>.json` with `passed` and the reasons. Thresholds: at least 1000 rows (less than one minute of traffic at 20 events/s means the stream was down that day); at most 1 % duplicated ids (Spark already removes real duplicates, but a simulator restart reuses ids for different events: 0.76 % after one restart in our first test); at most 5 % UNKNOWN (one unknown vehicle = 2.5 % of the fleet; more than two means the catalog is broken). On 2026-10-13: 980701 rows, 0.63 % reused ids, 0.01 % UNKNOWN, passed. **Daily report.** Per vehicle and per region: events, mean and max speed, speeding and harsh-braking events; the five worst vehicles by *rate* of incidents, otherwise the hot vehicle, which sends 40 % of the traffic, would always be last. **Partition pruning:** the plan shows `PartitionFilters: [isnotnull(event_date), (event_date = 2026-10-11)]` and Spark logs `Selected 1 partitions out of 589, pruned 99.83%`; only that file is read. Spark still listed all folders first (25 s): pruning saves reading, not listing.

Skew. With `--hot-vehicle`, V0001 = 37 % of the rows. The report aggregation itself shows no skew: Spark aggregates partially in each task before the shuffle (the combiner of Chapter 2), so each task sends at most one row per vehicle (tasks of 34-53 ms). To show skew we used a naive version that repartitions by vehicle first:

Aggregation stage, 8 tasks	965k rows: task ms / rows (min, median, max)	x10 rows (9.7M): task ms (min, median, max), stage
naive (repartition by vehicle)	271 / 279 / 394 ms; 47k / 78k / **402k**	407 / 599 / **1879** ms; stage 1888-1977 ms
naive + AQE	4 tasks, hot task still 402k rows	7 tasks, hot task still 4.03M rows, 1824 ms
salting (8 salts, 2 stages)	max/median rows 1.1	800 / 1206 / **1325** ms; stage 1338-1432 ms (-28 %)

Salting balances the tasks and cuts the aggregation stage by about 28 % at 10x volume; the whole query is not faster because salting adds a column, a shuffle and a second aggregation. AQE alone never splits the hot partition: it only merges small ones (its skew handling applies to joins and `rebalance()`). The x10 run repeats each row 10 times in memory (`--copies 10`): we did not have ten hours of 1000 events/s data, it only shows the stage with more rows per key.

6. Airflow (Part D)

fleet_daily_pipeline (@daily): `PythonSensor` in reschedule mode (every 60 s, 2 h timeout) waiting for a Parquet file of `{{ ds }}` -> `check_topics` (3 retries, 30 s, a broker restart is a typical transient error) -> `run_quality` -> `@task.branch` on the verdict -> either `run_report` then `publish_report` (to `daily_reports`) or `publish_quality_alert` -> `end` with `none_failed_min_one_success` (with the default `all_success` the skipped branch would skip it). Both paths were run: a real day and a test day with 30 rows; both ended in

success. **kafka_lag_monitor** (every 5 min): a consumer with `group.id=alerts-notifier` that never subscribes reads the committed offsets and the end offsets; lag = end - committed. It never joins the group, so it never causes a rebalance (checked in the notifier logs). Threshold 50 per partition: we measured ~10 alerts per minute per partition and a normal lag of 0, so 50 is about 5 minutes of unhandled alerts.

Logical date vs execution time. The run of logical date 2026-10-13 covers that day and starts at the end of the interval, 2026-10-14 00:00 UTC: the logical date says which data to process, not when. **catchup=False**: we only have data for the days the platform ran; with catchup every day since `start_date` would get a run whose sensor waits 2 h and fails. We saw it live: the laptop slept from Oct 14 to Oct 17 and, at wake-up, Airflow created only the run of the last interval. A **backfill of the last 7 days** would create 7 runs, each for its own `ds`: days with files get a verdict and a report (or an alert), days without files fail after the 2 h sensor timeout, and the report or alert of each day is published again. **Idempotence:** the sensor, the checks, the branch and both Spark jobs are idempotent (same date -> same files read -> JSON overwritten); the Kafka publications are not (a rerun appends a second message, mitigated by key = date). Proof: `run_quality` cleared and run twice on a frozen day gave the same file (same SHA-256). **Poke mode** would keep the worker slot during the whole wait. Airflow standalone uses the SequentialExecutor, one task at a time: a poking sensor for a day without data would have blocked the lag monitor and every other DAG for 2 hours. In reschedule mode it freed the slot between checks and the other runs went on.

7. Distributed-systems analysis

- **Where is the consensus?** Only in Kafka: the KRaft controller quorum (Raft, Chapter 1) elects the active controller and commits metadata to `__cluster_metadata` once a majority has it. With one combined broker/controller the quorum is 1 node: a majority of 1, so 2f+1 = 1 and f = 0 failures tolerated. Consumer groups use a coordinator broker and a group leader that computes the assignment, which is coordination, not consensus. Spark has no consensus: the driver is a single coordinator and a single point of failure, recovered by restart from checkpoints and by lineage for lost tasks. Airflow has one scheduler and a SQLite metadata database, no HA.
- **Replication.** RF = 1 everywhere (one broker): no ISR, `acks=all` means the leader alone, `min.insync.replicas` has no effect, and the quorum condition R + W > N is trivially met with N = 1 but tolerates no failure. Durability comes only from the broker's disk volume.
- **CAP / PACELC per component.** Kafka here is CP: during the broker outage (E3) no write was accepted; availability was given back at the application level by the producer's local buffer, paying latency (the E of PACELC: latency for consistency). The Parquet output is consistency-first (exactly-once by idempotent overwrite). The alerts path favours availability and progress: at-least-once, duplicates rather than losses. The KPIs in update mode are eventually consistent (a window is republished until it closes). The Airflow quality gate chooses consistency: no report rather than a wrong one.
- **Resource model (Chapter 2).** Spark runs in local mode inside a container: driver and executor threads share one JVM and the container's cores, Docker plays the role of YARN's NodeManager containers and Compose the role of the ResourceManager. Two consequences we measured: the four queries compete for the same cores (Section 8), and local mode does not retry failed tasks, which made the job die in E3. The Parquet hand-off sits on a Windows bind mount, which behaves like a file system with a very high per-file cost: the small-files problem that HDFS avoids with 128 MB blocks.

8. Experiments and performance study (Part E)

All runs on the student laptop (i7-13700H, 20 threads, Docker limited to 16 GB). Each experiment has a hypothesis written before the run, raw outputs and observations in `experiments/`. No event counts below come from the Spark UI alone: we recounted independently what entered Kafka (same validation rules in plain Python) and compared it with the Parquet, the DLQ and the alerts.

Experiment	Hypothesis	Observed	Explanation
E1 kill -9 of the streaming job, 52 s down	nothing lost, Parquet intact, a few duplicated alerts	0 lost, 0 duplicated in Parquet, DLQ and alerts; the in-flight batch replayed with the same 140 records, then a 2164-record catch-up batch	the kill fell before the batch wrote anything; duplicates need a kill between alert send and commit (shown by a forced replay: alerts republished, Parquet unchanged)

Experiment	Hypothesis	Observed	Explanation
E2 notifiers stopped 10 min	lag +10/min per partition, alert after ~5-10 min, fast catch-up	lag up to 93/127/118, `consumer_lag_high` published at +5.5 and +10.5 min, lag 0 ten seconds after restart	offsets and alerts are kept on the broker; the monitor samples every 5 min and its alert lands in the lagging topic
E3 broker stopped 65 s	no loss for the producer, Spark waits and continues	producer: 1305 messages created during the outage all delivered, no gap in sequence numbers; **Spark job died** after 59 s; notifiers rejoined alone	Kafka client timeout 60 s < 65 s, no task retry in local mode, one failed query stops the app; restarted from checkpoint without loss
E4 burst 10x (200 events/s)	Kafka fine, Spark batches longer, latency up	q1 batch ~7 s for 150 or 1600 records (also ~7 s at 1000 events/s), latency median 13.7 s -> 11.2 s	the fixed cost per micro-batch dominates; the bottleneck is per-batch overhead, not volume

Global check after E1-E4: of 1035521 messages, all 10240 invalid ones are in the DLQ; 26844 point alerts, none duplicated or missing. 19995 valid events are missing because of the watermark; 203 more were dropped by the dedup because a restarted simulator reused the ids of events still in the dedup state (all 203 have the same id as an event 9 to 251 s away). Our dedup does what was asked; the ids are not unique across simulator restarts.

Run (1.08M-event backlog, maxOffsetsPerTrigger 100k)	Catch-up throughput	q1 steady batch	Alert latency median / p95
shuffle.partitions 6, default trigger	7503 records/s	6.7-8.1 s	12.4 s / 17.0 s
shuffle.partitions 6, trigger 10 s	8585 records/s	5.6-6.8 s	10.7 s / 15.0 s
shuffle.partitions 200, default trigger	229 records/s (7 min per batch)	never reached	not defined (backlog never drained)
shuffle.partitions 200, trigger 10 s	cancelled (could not drain either)		

With 200 partitions, q1 writes 200 Parquet files (plus 200 checksum files) per batch through the bind mount, about one second each; the Kafka-only queries q2 and q3 drained the same backlog in 4 minutes. A 10 s trigger was not worse, against our hypothesis: the four queries no longer run back-to-back on the same cores, each q1 batch is shorter, and the wait for the next trigger is compensated (differences of ~1.5 s measured on one 3-min window each). **Recommended:** 6 shuffle partitions (= Kafka partitions), a 10 s trigger (fewer batches, so fewer small files for the batch jobs) and `maxOffsetsPerTrigger` to bound catch-up batches after a crash.

9. Difficulties

- Docker networking on Windows: the provided broker was unreachable from other containers until we added a second listener.
- Two silent traps in Spark: `from_json` returning a struct of NULLs on broken JSON, and NULL comparisons hiding messages from both the valid and the invalid branch.
- The notifier crashed on a commit refused during a rebalance; we only saw it on the laptop, with real alert traffic.
- The skew was invisible at first because of the partial aggregation; finding why took longer than implementing salting.
- E3 contradicted our hypothesis: understanding from the logs that a single 60 s timeout stops all four queries.
- Accounting: proving "no loss" required our own recount of everything in Kafka, which revealed the id collisions.
- Performance of the Windows bind mount with thousands of small Parquet files (8 min for a daily pipeline on one day).

10. Limits and what we would do with one more month

- One broker, RF = 1, one controller: no fault tolerance at all. -> 3 brokers and 3 KRaft controllers, RF = 3, `min.insync.replicas=2`, `acks=all`.
- Spark in local mode, no supervision: one Kafka timeout kills the job. -> run on Kubernetes or YARN with restart on failure (safe thanks to the checkpoints), task retries, a longer Kafka client timeout.

- At-least-once alerts and repeated sustained alerts. -> deduplicate in the notifier on `event_id` / window, or a transactional producer for an exactly-once read-process-write path.
- Small files and catalog re-read at every batch. -> Delta Lake or Iceberg with compaction instead of one folder per batch, catalog cached and refreshed every few minutes.
- Late events dropped by the 2-minute watermark, ids not unique across simulator restarts. -> watermark sized on the measured lateness (about 6 min), producer-unique ids.
- Airflow on SQLite with the SequentialExecutor, alerts sent to the topic they monitor, no TLS, SASL or ACLs. -> PostgreSQL and LocalExecutor, a second alert channel, Kafka security from day one (Chapter 4).

11. Work split and contributions

We split the work along the architecture: the Parquet hand-off files are the boundary between the real-time path and the batch / orchestration path. Each of us owned one side end to end, including the experiments that stress it; the interface between the two (topic names and keys, Parquet layout `event_date/batch_id`, verdict format) was decided together.

	Nathan CHEVAUCHE	**Benjamin TONNEAU**
Owned	Docker Compose infrastructure; Part A (catalog publisher, notifier, keys, partitions, semantics); Part B (streaming job, DLQ, dedup, enrichment, three queries, checkpoints)	Part C (quality job, daily report, partition pruning, skew study); Part D (both DAGs, idempotence, logical date / catchup / backfill); measurement scripts
Experiments	E1 (kill -9), E3 (broker down), Part A and B demos	E2 (notifier down / lag monitor), E4 (burst), performance study
Together	architecture choices, global data check, report, defence	architecture choices, global data check, report, defence

Why this split: each side can be built and tested alone (the batch side only needs Parquet files, the streaming side only needs Kafka), it balances the grading weight (A+B real time, C+D+E measurement and orchestration), and each of us can defend his whole side, including how it fails.

12. Sources and tools

Course material (Chapters 1-4, practical works 1-5, study guide); official documentation of Apache Spark (Structured Streaming guide), Apache Kafka, confluent-kafka-python, Apache Airflow and Docker; the provided Compose file, topic script and simulator (changes listed in Section 1).